

Claude Code 4월 업데이트 후 tmux 불가 문제 - WSL2·Git Bash 전환으로 해결하기

"어? tmux가 갑자기 안 되네... 🤔"

2026년 4월 12일 무렵 Claude Code 업데이트 이후, 여기저기서 이런 이야기가 들려오기 시작했습니다.

사실 Anthropic이 PowerShell 툴의 보안을 대폭 강화하면서, Windows의 PowerShell에서 tmux를 사용한 페인 분할과 에이전트 팀 기능이 사실상 차단되기 시작한 것입니다 🤯

"Windows Terminal에서 쓰던 게 안 되는 건가요?" "어떻게 해야 하나요?"라고 당황하신 분들도 많을 것 같습니다.

안심하세요. 해결책은 있습니다. WSL2 또는 Git Bash를 기본 셸로 설정해 Claude Code를 실행하는 방식으로 전환하기만 하면, tmux를 다시 제대로 쓸 수 있게 됩니다 ✨

🔴 무슨 일이 벌어진 건가? - PowerShell과 tmux의 관계

먼저 상황을 정리해 보겠습니다.

Claude Code는 Windows에서 tmux를 사용할 때, 내부적으로 `process.stdout.isTTY` 라는 값을 확인해 터미널 환경을 판단합니다. 그런데 Claude Code가 사용하는 실행 환경인 **Bun SFE**에서는, Windows에서 이 값이 항상 `undefined` (미정의)가 되어버리는 알려진 버그가 존재했습니다 (GitHub Issue [#26244](#)).

이전부터 문제는 있었지만, 이번 업데이트로 PowerShell 툴의 보안이 더욱 강화되면서 상황이 더 심각해졌습니다. 구체적으로는:

- 백그라운드 잡(Job)을 통한 우회 경로를 막는 처리가 강화됨
- -ErrorAction Break 로 인한 디버거 행(hang) 현상 수정
- Expand-Archive 등 압축 해제 명령의 경로 안전성 검사 강화
- 명령 인자 파싱 실패 시 폴백 규칙이 "거부"로 변경됨

쉽게 말해, 보안 나사가 한층 더 조여진 것입니다 🛡️

그 영향으로 PowerShell에서 tmux의 페인 분할 명령(`split-window` , `send-keys` 등)을 호출하는 처리가 전부 차단되기에 이르렀습니다.

🟡 왜 WSL2 / Git Bash에서는 동작하나?

"그럼 왜 WSL2나 Git Bash에서는 되는 거죠?"라는 의문은 당연합니다.

WSL2(Windows Subsystem for Linux 2) 안에서는 Linux 네이티브 tmux 바이너리가 그대로 동작합니다. `process.stdout.isTTY` 문제도 없고, Claude Code가 터미널 환경을 정상적으로 인식해 tmux 모드로 전환됩니다.

Git Bash의 경우도 마찬가지로, MINGW라는 Linux 호환 레이어를 통해 tmux 동작 환경이 갖춰집니다.

실제로 GitHub에는 Windows 네이티브에서 tmux를 동작시키기 위한 **psmux**라는 Rust 제작 툴도 등장했습니다. 76개의 tmux 명령에 대응하며, PowerShell 7.5, `cmd.exe`, Git Bash, WSL 모두를 동일 세션에서 다룰 수 있는 우수한 툴입니다. 다만 아직 Claude Code와의 공식 통합은 개발 중이므로, 현시점에서는 공식 지원이 있는 **WSL2/Git Bash**를 사용하는 것이 가장 확실합니다.

✅ 지금 당장 할 수 있다! WSL2 / Git Bash 전환 방법

① WSL2를 기본 셸로 설정하는 경우

절차:

1. Windows 터미널을 열기
2. 「설정」→「시작」
3. 「기본 프로필」을 **Ubuntu(또는 원하는 Linux 배포판)** 로 변경
4. WSL2 터미널에서 다음을 실행:

```
# Claude Code 실행
claude
```

[보완] Windows 터미널에서 WSL2 프로필이 보이지 않는 경우, 먼저 PowerShell(관리자)에서 `wsl --install` 을 실행해 WSL2를 설치해야 합니다. 이미 설치되어 있다면 `wsl --list --verbose` 로 설치된 배포판을 확인하세요.

5. tmux 세션을 먼저 열고 Claude Code를 실행:

```
tmux new-session -s dev
claude
```

이것으로 에이전트 팀의 페인 분할 기능이 복구됩니다 🎉

[보완] 한국 개발 환경에서 Windows Terminal은 Microsoft Store에서 무료로 설치할 수 있습니다. WSL2 Ubuntu 배포판 설치 후 터미널을 재시작하면 드롭다운 메뉴에 Ubuntu 프로필이 자동으로 추가됩니다.

② Git Bash를 기본 셸로 설정하는 경우

Git for Windows가 설치되어 있는 분들에게는 이 방법이 간편합니다.

1. Windows 터미널 설정에서 「Git Bash」를 기본 프로필로 설정
2. 또는 직접 Git Bash를 실행

3. tmux가 설치되어 있다면:

```
tmux new-session -s mywork  
claude
```

으로 동작합니다.

Git Bash에서의 tmux는 MSYS2를 통해 설치할 수 있습니다:

```
pacman -S tmux
```

[보완] Windows 터미널의 설정 JSON에서 Git Bash 프로파일을 수동으로 추가하려면, `commandline` 필드에 `C:\\Program Files\\Git\\bin\\bash.exe` 를 지정하세요. Git for Windows는 <https://git-scm.com> 에서 다운로드할 수 있습니다.

Claude Code tmux 설정 복습

이 기회에, tmux를 올바르게 사용하기 위한 설정도 확인해 두겠습니다.

settings.json에서의 tmux 설정 예시:

```
{  
  "env": {  
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"  
  }  
}
```

tmux 세션 내에서 Agent Teams 활성화:

```
# 세션 시작
```

```
tmux new-session -s claude-dev
```

```
# Claude Code 실행
```

```
CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1 claude
```

이 환경 변수를 설정하면 tmux 페인 분할을 활용한 멀티 에이전트 팀 기능이 활성화됩니다.

[보완] settings.json 의 위치는 ~/.claude/settings.json (WSL2/Git Bash 환경 기준)입니다. Claude Code 설치 후 `claude config` 를 실행하면 설정 파일 위치를 확인할 수 있습니다.

🔮 향후 전망 - psmux라는 희망의 빛

조금 앞의 이야기를 하자면, 실은 "psmux로 Windows 네이티브 대응을 실현하자"는 GitHub Issue(#34150)가 올라와 있어 커뮤니티에서 활발하게 논의되고 있습니다.

psmux는 Rust 제작의 Windows 네이티브 tmux 구현으로, WSL 없이도 tmux 명령을 사용할 수 있는 툴입니다. Claude Code의 감지 로직에 작은 변경을 가하기만 하면 통합할 수 있다고 하며, 장래적으로는 WSL2 없이 PowerShell에서 tmux를 쓸 수 있게 될지도 모릅니다.

다만 2026년 4월 현재로서는 실험적인 단계입니다. 공식 지원이 갖춰질 때까지는 **WSL2/Git Bash에서 Claude Code를 실행하는 것이 가장 안정적인 해결책입니다** ✨

✨ 정리

이번 변경 내용을 정리하면 다음과 같습니다.

- Claude Code 4월 업데이트로 PowerShell 툴의 보안이 강화됨
- 그 영향으로 PowerShell에서의 tmux 조작이 차단되는 상태가 됨
- 근본 원인은 Bun SFE on Windows의 `process.stdout.isTTY` 버그 (Issue

#26244)

- 지금 당장의 대처법: WSL2 또는 Git Bash를 기본 셸로 설정해 Claude Code를 실행한다
- Windows 터미널에서는 기본 프로필 설정을 WSL2/Git Bash로 변경하기만 하면 OK
- 장래적으로는 psmux 통합을 통한 Windows 네이티브 대응이 기대됨

"또 사양 변경이야... 😓"라는 마음은 충분히 이해합니다. 하지만 보안이 강화됐다는 것은 그만큼 툴이 성숙해지고 있다는 의미이기도 합니다.

한 번 WSL2를 메인으로 삼아버리면, Linux 환경의 혜택으로 오히려 개발 경험이 올라가는 경우도 많습니다. 이 기회에 꼭 WSL2 환경을 갖춰보세요 💪

Jaewoo Kim @aiengineer